

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 05 -

SEGURANÇA E BOAS PRÁTICAS EM APIS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	28
REFERÊNCIAS.....	29

EMSE

O QUE VEM POR AÍ?

Imagine que você tenha desenvolvido uma API para servir previsões de um modelo de Machine Learning. Ela está no ar e qualquer cliente pode fazer requisições, mas como garantir que apenas usuários autorizados acessem o serviço, que os dados de entrada não causem falhas ou brechas de segurança e que sua API permaneça estável mesmo sob uso intenso?

Nesta aula, vamos explorar práticas essenciais de segurança e robustez em APIs desenvolvidas com FastAPI, cobrindo desde autenticação e autorização com tokens JWT, passando por validação de entrada de dados, controle de acesso via CORS e medidas de proteção contra abusos (rate limiting), até boas práticas gerais como logging, uso de HTTPS, manejo de credenciais e, por fim, considerações de produção (escalabilidade, redundância, alta disponibilidade). Tudo isso com exemplos práticos em código para consolidar o aprendizado.

HANDS ON

Prepare-se para mais uma aula hands on, onde vamos melhorar nossa API FastAPI e explorar práticas essenciais de segurança e robustez em APIs desenvolvidas com FastAPI! Abra seu VSCode e assista os vídeos para auxiliar na parte prática!



SAIBA MAIS

Autenticação e Autorização (Tokens JWT)

Em segurança de APIs, autenticação é o processo de verificar quem é o usuário ou sistema que faz a requisição, enquanto a autorização determina o que esse usuário pode fazer. Em aplicações web modernas, especialmente APIs REST stateless, é comum usar tokens de acesso para essas tarefas, em vez de sessões tradicionais.

Um dos padrões mais difundidos é o JWT (JSON Web Token), um token em formato JSON, assinado criptograficamente, que carrega claims como usuário, permissões, validade etc. que o servidor pode verificar sem precisar de armazenamento de sessão.



Figura 1 – JWT
Fonte: Google Imagens (2026)

No FastAPI, podemos proteger endpoints exigindo um token JWT válido; é uma forma de codificar o objeto JSON em uma string longa e densa sem espaços, como:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MzE1MjM1MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P  
Ok6yJV_adQssw5c
```

Existem bibliotecas e componentes prontos como OAuth2PasswordBearer, mas vamos ilustrar aqui um exemplo simples utilizando a biblioteca PyJWT para decodificar e verificar o token manualmente.

Pensando na nossa API Iris da aula anterior, criamos um endpoint /predic que usava nosso modelo e fazia a previsão do tipo de flor. Isso é um problema em produção pois qualquer pessoa poderia chamar essa rota, tendo muito provavelmente custos descontrolados pois cada chamada utiliza recursos computacionais, dados sensíveis poderiam ser expostos e principalmente nossa API teria vulnerabilidade a ataques.

É importante termos em mente esses dois conceitos fundamentais:

CONCEITO	PERGUNTA	EXEMPLO
Autenticação	QUEM é você?	Login com usuário/senha
Autorização	O que você PODE fazer?	Admin pode retreinar, usuário apenas faz predição

Tabela 1 - Autenticação vs Autorização
Fonte: Elaborado pelo autor (2026)

Formas comuns de autenticação em APIs:

- **API Key:** Token estático enviado no header:
 - Simples de implementar.
 - Bom para comunicação entre serviços.
- **JWT (JSON Web Token):** Token assinado com informações do usuário:
 - Mais seguro.
 - Pode expirar automaticamente.

- **OAuth2:** Protocolo completo para autenticação:
 - Usado por Google, Facebook etc.
 - Mais complexo.

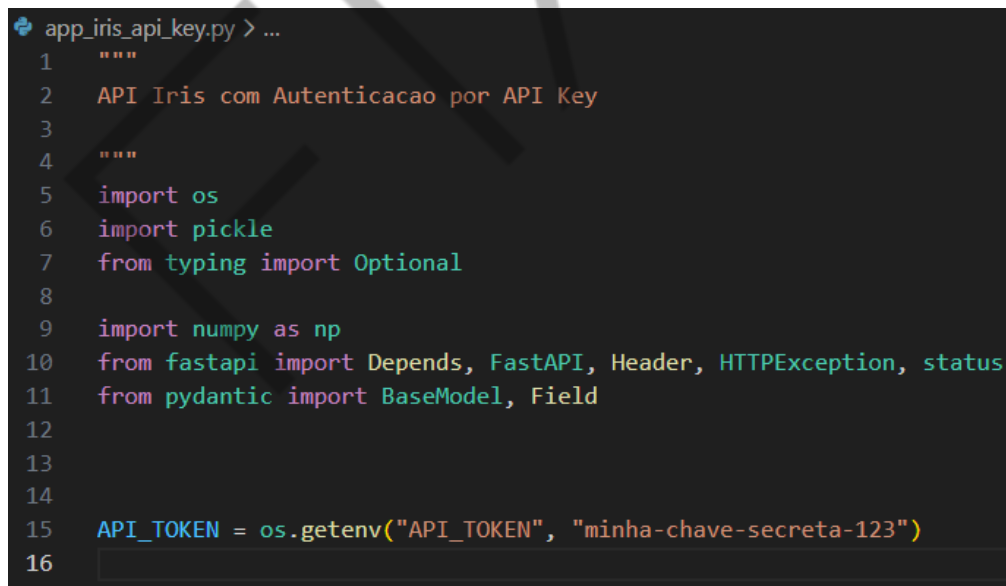
Vamos implementar as duas primeiras na nossa API Iris.

Hands On - API Key

Vamos criar uma versão da API Iris que exige uma API Key no header X-API-Key.

Fluxo:

- Cliente envia requisição com HEADER X-API-Key: <token>.
- API valida se o token está correto.
- Se válido, processa a predição.
- Se inválido, retorna erro 401 (Unauthorized).



```
app_iris_api_key.py > ...
1  """
2  API Iris com Autenticacao por API Key
3
4  """
5  import os
6  import pickle
7  from typing import Optional
8
9  import numpy as np
10 from fastapi import Depends, FastAPI, Header, HTTPException, status
11 from pydantic import BaseModel, Field
12
13
14
15 API_TOKEN = os.getenv("API_TOKEN", "minha-chave-secreta-123")
16
```

Figura 2 – API-KEY
Fonte: Elaborado pelo autor (2026)

Importante reforçar: em produção, use variáveis de ambiente! Nunca deixe tokens no código!

```
class IrisRequest(BaseModel):
    sepal_length: float = Field(..., ge=0, le=10, description="Comprimento da sepala (cm)")
    sepal_width: float = Field(..., ge=0, le=10, description="Largura da sepala (cm)")
    petal_length: float = Field(..., ge=0, le=10, description="Comprimento da petala (cm)")
    petal_width: float = Field(..., ge=0, le=10, description="Largura da petala (cm)")

class IrisResponse(BaseModel):
    sucesso: bool
    classe: str
    classe_idx: int
    probabilidades: dict
```

Figura 3 – Schemas Pydantic
Fonte: Elaborado pelo autor (2026)

Os esquemas são os mesmos da aula anterior. Agora, vamos criar uma async def para validação da API Key.

```
async def verify_api_key(x_api_key: Optional[str] = Header(default=None)):
    """
    Dependência que valida o header X-API-Key.
    Se a chave estiver errada ou ausente, retorna HTTP 401.
    """
    if x_api_key is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="API Key ausente. Envie o header X-API-Key.",
        )
    if x_api_key != API_TOKEN:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="API Key inválida.",
        )
    return x_api_key
```

Figura 4 – Async Def API Key
Fonte: Elaborado pelo autor (2026)

Agora, vamos carregar o modelo:

```
try:
    with open('modelo_iris.pkl', 'rb') as f:
        modelo = pickle.load(f)
    with open('classes_iris.pkl', 'rb') as f:
        classes = pickle.load(f)
    MODELO_CARREGADO = True
except FileNotFoundError:
    modelo = None
    classes = None
    MODELO_CARREGADO = False
```

Figura 5 – Carregamento do Modelo
Fonte: Elaborado pelo autor (2026)

```
app = FastAPI(
    title="API Iris com Autenticacao",
    description="API de classificacao Iris protegida por API Key",
    version="2.0.0",
)

@app.get("/")
def home():
    """Endpoint publico - informacoes da API."""
    return {
        "api": "Iris Classifier",
        "versao": "2.0 (com API Key)",
        "modelo_carregado": MODELO_CARREGADO,
        "endpoints": {
            "/": "Publico - info da API",
            "/health": "Publico - status",
            "/predict": "PROTEGIDO - requer X-API-Key",
        }
    }

@app.get("/health")
def health():
    """Endpoint publico - health check."""
    return {"status": "healthy" if MODELO_CARREGADO else "unhealthy"}
```

Figura 6 – API e rotas principais
Fonte: Elaborado pelo autor (2026)

O nosso endpoint POST /predict vai ser protegido, exigindo header X-API-Key válido:

```
@app.post("/predict", response_model=IrisResponse, dependencies=[Depends(verify_api_key)])
def predict(payload: IrisRequest):
    """
    Endpoint PROTEGIDO - requer header X-API-Key valido.
    """
    if not MODELO_CARREGADO:
        raise HTTPException(status_code=503, detail="Modelo nao carregado")

    features = np.array([[
        payload.sepal_length,
        payload.sepal_width,
        payload.petal_length,
        payload.petal_width
    ]])

    predicao_idx = modelo.predict(features)[0]
    probabilidades = modelo.predict_proba(features)[0]

    return IrisResponse(
        sucesso=True,
        classe=classes[predicao_idx],
        classe_idx=int(predicao_idx),
        probabilidades={
            classes[i]: round(float(p), 4)
            for i, p in enumerate(probabilidades)
        }
    )
```

Figura 7 – Endpoint Predict
Fonte: Elaborado pelo autor (2026)

Se tentarmos acessar o endpoint sem colocar a nossa API-KEY, ele vai dar o erro 401:



Figura 8 – Swagger 401
Fonte: Elaborado pelo autor (2026)

```
INFO: 127.0.0.1:54139 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:65125 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:65125 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:55665 - "POST /predict HTTP/1.1" 401 Unauthorized
INFO: 127.0.0.1:51417 - "POST /predict HTTP/1.1" 401 Unauthorized
```

Figura 9 – Terminal 401
Fonte: Elaborado pelo autor (2026)

Ao colocar a API-KEY, conseguimos acessar esse endpoint e fazer uma previsão:

POST /predict Predict

Endpoint PROTEGIDO - requer header X-API-Key valido.

Parameters

Name	Description
X-api-key	string (string null) (header)

Request body required application/json

Figura 10 – Predict 200 (1)
Fonte: Elaborado pelo autor (2026)

Server response

Code	Details
200	Response body <pre>{ "sucesso": true, "classe": "virginica", "classe_id": 0, "probabilidades": { "setosa": 0, "versicolor": 0, "virginica": 1 } }</pre> Response headers <pre>content-length: 117 content-type: application/json date: Tue, 09 Dec 2025 13:22:21 GMT server: uvicorn</pre>

Figura 11 – Predict 200 (2)
Fonte: Elaborado pelo autor (2026)

API Iris agora com JWT

O JWT é mais seguro que a nossa API-Key por que ele pode expirar automaticamente, contém informações do usuário que fez o login e tem assinatura digital, não sendo possível falsificá-la.

O fluxo é:

- Usuario faz login com usuario/senha em `/login`.
- API retorna um token JWT.
- Usuario envia o token no header `Authorization: Bearer <token>`.
- API valida o token e processa a requisição.

Para começarmos, vamos instalar o PyJWT e ajustar a API.:

```
import os
import pickle
from datetime import datetime, timedelta
from typing import Optional

import jwt
import numpy as np
from fastapi import Depends, FastAPI, Header, HTTPException, status
from pydantic import BaseModel, Field

SECRET_KEY = os.getenv("JWT_SECRET", "minha-chave-jwt-super-secreta")
ALGORITHM = "HS256"
TOKEN_EXPIRE_MINUTES = 30

# Usuario fake para demonstracao (em producao, use banco de dados!)
USERS_DB = {
    "admin": {"password": "admin123", "role": "admin"},
    "user": {"password": "user123", "role": "user"},
}
```

Figura 12 – JWT
Fonte: Elaborado pelo autor (2026)

Também atualizamos nossos SCHEMAS:

```
class LoginRequest(BaseModel):
    username: str
    password: str

class TokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
    expires_in: int = TOKEN_EXPIRE_MINUTES * 60

class IrisRequest(BaseModel):
    sepal_length: float = Field(..., ge=0, le=10)
    sepal_width: float = Field(..., ge=0, le=10)
    petal_length: float = Field(..., ge=0, le=10)
    petal_width: float = Field(..., ge=0, le=10)

class IrisResponse(BaseModel):
    sucesso: bool
    classe: str
    probabilidades: dict
    usuario: str # Novo: mostra quem fez a requisicao
```

Figura 13 – SCHEMAS
Fonte: Elaborado pelo autor (2026)

Criamos também funções de JWT para criar token e validar usuário:

```
def create_token(username: str, role: str) -> str:
    """Cria um token JWT com expiracao."""
    expire = datetime.utcnow() + timedelta(minutes=TOKEN_EXPIRE_MINUTES)
    payload = {
        "sub": username,      # subject = quem e o usuario
        "role": role,        # papel do usuario
        "exp": expire,        # quando expira
    }
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

def get_current_user(authorization: Optional[str] = Header(default=None)) -> dict:
    """Extrai e valida o usuario do token JWT."""
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token ausente. Use: Authorization: Bearer <token>",
        )

    token = authorization.split(" ", 1)[1]

    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return {"username": payload["sub"], "role": payload["role"]}
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="Token expirado. Faca login novamente.")
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail="Token invalido.")
```

Figura 14 – JW-3
Fonte: Elaborado pelo autor (2026)

Na sequência a parte de carregamento do modelo fica igual, mas vamos fazer alguns ajustes nas rotas:

```
app = FastAPI(
    title="API Iris com JWT",
    version="2.0.0",
    openapi_tags=[
        {"name": "Auth", "description": "Autenticação"},
        {"name": "Predict", "description": "Predição protegida"},
    ]
)

@app.get("/")
def home():
    """Endpoint publico - informacoes da API."""
    return {
        "api": "Iris Classifier",
        "versao": "3.0 (com JWT)",
        "modelo_carregado": MODELO_CARREGADO,
        "endpoints": {
            "/": "Publico - info da API",
            "/login": "Publico - login para obter token JWT",
            "/me": "PROTEGIDO - informacoes do usuario autenticado",
            "/predict": "PROTEGIDO - predicao de classe Iris (requer JWT)",
        }
    }
```

Figura 15 – JW-4
Fonte: Elaborado pelo autor (2026)

```
@app.get("/")
def home():
    """Endpoint publico - informacoes da API."""
    return {
        "api": "Iris Classifier",
        "versao": "3.0 (com JWT)",
        "modelo_carregado": MODELO_CARREGADO,
        "endpoints": {
            "/": "Publico - info da API",
            "/login": "Publico - login para obter token JWT",
            "/me": "PROTEGIDO - informacoes do usuario autenticado",
            "/predict": "PROTEGIDO - predicao de classe Iris (requer JWT)",
        }
    }

@app.post("/login", response_model=TokenResponse, tags=["Auth"])
def login(credentials: LoginRequest):
    """Faz login e retorna um token JWT."""
    user = USERS_DB.get(credentials.username)

    if not user or user["password"] != credentials.password:
        raise HTTPException(status_code=401, detail="Usuario ou senha incorretos")

    token = create_token(credentials.username, user["role"])
    return TokenResponse(access_token=token)
```

Figura 16 – JW-5
Fonte: Elaborado pelo autor (2026)

```
@app.get("/me", tags=["Auth"])
def get_me(current_user: dict = Depends(get_current_user)):
    """Retorna informacoes do usuario logado."""
    return {
        "username": current_user["username"],
        "role": current_user["role"],
        "message": "Voce esta autenticado!"
    }
```

Figura 17 – JW-6
Fonte: Elaborado pelo autor (2026)

```
@app.post("/predict", response_model=IrisResponse, tags=["Predict"])
def predict(payload: IrisRequest, current_user: dict = Depends(get_current_user)):
    """Predicao protegida por JWT."""
    if not MODELO_CARREGADO:
        raise HTTPException(status_code=503, detail="Modelo nao carregado")

    features = np.array([[
        payload.sepal_length, payload.sepal_width,
        payload.petal_length, payload.petal_width
    ]])

    pred_idx = modelo.predict(features)[0]
    probs = modelo.predict_proba(features)[0]

    return IrisResponse(
        sucesso=True,
        classe=classes[pred_idx],
        probabilidades={classes[i]: round(float(p), 4) for i, p in enumerate(probs)}
        usuario=current_user["username"] # Mostra quem fez a requisicao
    )
```

Figura 18 – JW-7
Fonte: Elaborado pelo autor (2026)

Agora, podemos acessar a documentação swagger e, através da rota POST, fazer login:

POST

/login Login

Faz login e retorna um token JWT.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{  "username": "admin",  "password": "admin123"}  
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  "http://127.0.0.1:8000/login" \  -H 'accept: application/json' \  -H 'content-type: application/json' \  -d '{  "username": "admin",  "password": "admin123"}'
```

Request URL

http://127.0.0.1:8000/login

Server response

Code

Details

200

Response body

```
{  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ2bm90aWZlIjpbZG1pbiIsInZyb2UiOiJhZG1pbiIsImV4cCI6MTYwMjYwOTQwLCJpdiI6ImVudy1qFk1ZD0mQ",  "token_type": "bearer",  "expires_in": 3600}
```

Download

Figura 19 – JW-6
Fonte: Elaborado pelo autor (2026)

Com o nosso `access_token` em mãos, na parte superior da documentação colocamos a autorização para acessar as rotas protegidas:

Available authorizations

HTTPBearer (http, Bearer)

Value:

zrWnhYVu0y-qFXiWZHDnmQ

Authorize

Close

Figura 20 – JW-7
Fonte: Elaborado pelo autor (2026)

E agora sim podemos acessar nossa API com mais segurança!

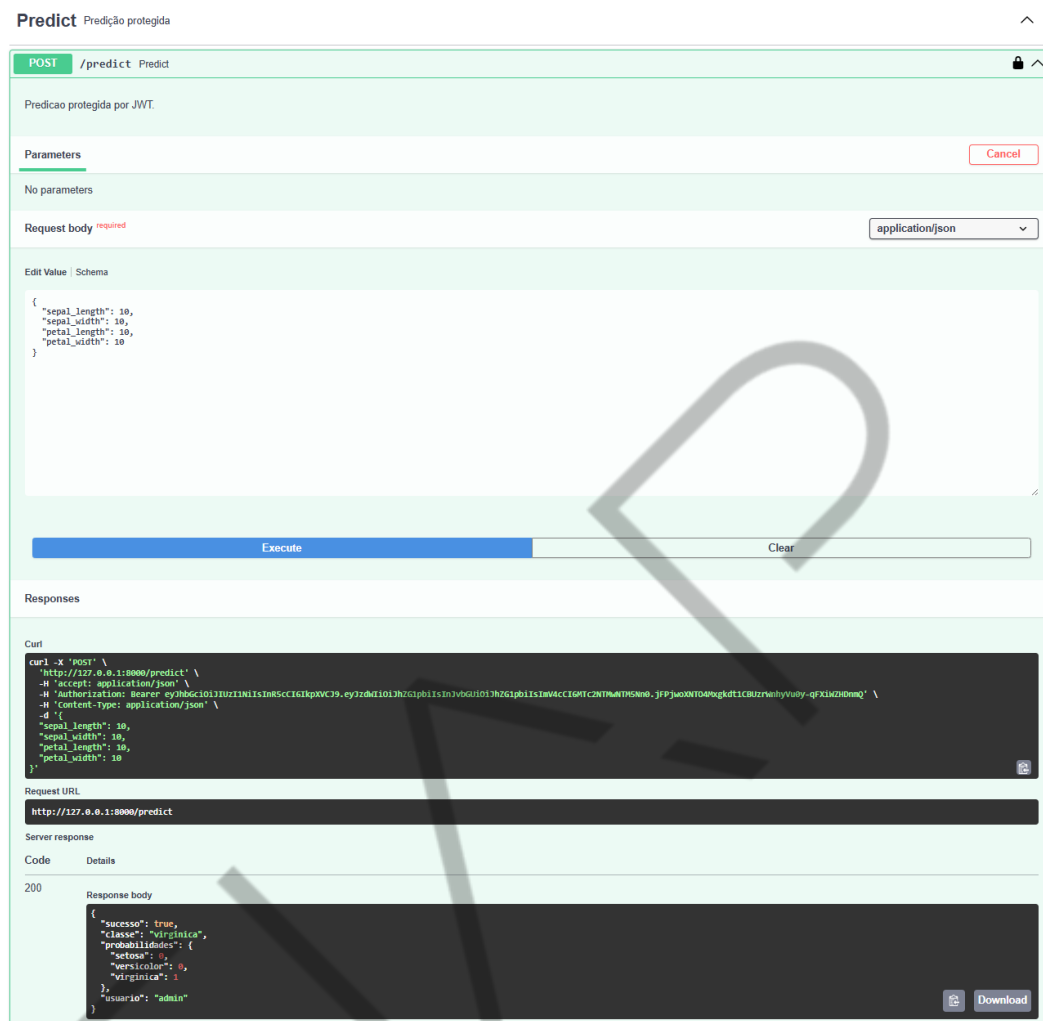


Figura 21 – JW-8
Fonte: Elaborado pelo autor (2026)

Rate Limiting - Protegendo contra Abusos

Mesmo em rotas protegidas, sem um limite de requisições, um atacante pode derrubar a nossa API com milhares de chamadas ou até mesmo consumir uma quantidade absurda de recursos GPU/CPU, gerando custos extras na cloud. Para nos ajudar com isso, o Rate Limiting limita X requisições por Y segundos por cliente. Por exemplo, um usuário do mesmo IP pode fazer no máximo 10 requisições por minuto.

```
REQUEST_LIMIT = 10      # Maximo de requisicoes
WINDOW_SECONDS = 60     # Janela de tempo (1 minuto)

# Armazena timestamps de requisicoes por IP
request_history = defaultdict(deque)
```

Figura 22 – Rate Limiting - 1
Fonte: Elaborado pelo autor (2026)

O carregamento do modelo fica igual, mas vamos mudar nossa aplicação com um middleware.

```
@app.middleware("http")
async def rate_limiter(request: Request, call_next):
    """Middleware que limita requisicoes por IP."""
    client_ip = request.client.host
    now = time.time()
    history = request_history[client_ip]

    # Remove requisicoes antigas (fora da janela)
    while history and now - history[0] > WINDOW_SECONDS:
        history.popleft()

    # Verifica se passou do limite
    if len(history) >= REQUEST_LIMIT:
        return JSONResponse(
            status_code=429,
            content={
                "detail": f"Limite excedido: {REQUEST_LIMIT} requisicoes por {WINDOW_SECONDS}s",
                "retry_after": int(WINDOW_SECONDS - (now - history[0]))
            }
        )

    # Registra esta requisicao
    history.append(now)

    # Continua processando
    response = await call_next(request)

    # Adiciona header informando quantas requisicoes restam
    response.headers["X-RateLimit-Remaining"] = str(REQUEST_LIMIT - len(history))
    response.headers["X-RateLimit-Limit"] = str(REQUEST_LIMIT)

    return response
```

Figura 23 – Rate Limiting - 2
Fonte: Elaborado pelo autor (2026)

```
@app.get("/")
def home():
    return {"api": "Iris com Rate Limiting", "limite": f"{REQUEST_LIMIT} req/{WINDOW_SECONDS}s"}

@app.post("/predict", response_model=IrisResponse)
def predict(payload: IrisRequest):
    if not MODELO_CARREGADO:
        raise HTTPException(status_code=503, detail="Modelo nao carregado")

    features = np.array([[
        payload.sepal_length, payload.sepal_width,
        payload.petal_length, payload.petal_width
    ]])

    pred_idx = modelo.predict(features)[0]
    probs = modelo.predict_proba(features)[0]

    return IrisResponse(
        sucesso=True,
        classe=classes[pred_idx],
        probabilidades={classes[i]: round(float(p), 4) for i, p in enumerate(probs)}
    )
```

Figura 24 – Rate Limiting - 3
Fonte: Elaborado pelo autor (2026)

Execute esse programa e teste via swagger. Ao clicar diversas vezes no endpoint de predição, quando extrapolamos o nosso limite de requisições, ele retorna o erro 429 Too Many Requests.



Figura 25 – Rate Limiting – 4
Fonte: Elaborado pelo autor (2026)

```
INFO:      Application startup complete.
INFO:      127.0.0.1:64958 - "GET / HTTP/1.1" 200 OK
INFO:      127.0.0.1:64958 - "GET / HTTP/1.1" 200 OK
INFO:      127.0.0.1:64958 - "GET /docs HTTP/1.1" 200 OK
INFO:      127.0.0.1:64958 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:      127.0.0.1:64958 - "POST /predict HTTP/1.1" 200 OK
INFO:      127.0.0.1:64958 - "POST /predict HTTP/1.1" 200 OK
INFO:      127.0.0.1:64958 - "POST /predict HTTP/1.1" 200 OK
INFO:      127.0.0.1:64958 - "POST /predict HTTP/1.1" 200 OK
INFO:      127.0.0.1:64958 - "POST /predict HTTP/1.1" 200 OK
INFO:      127.0.0.1:64958 - "POST /predict HTTP/1.1" 200 OK
INFO:      127.0.0.1:64958 - "POST /predict HTTP/1.1" 429 Too Many Requests
INFO:      127.0.0.1:64958 - "POST /predict HTTP/1.1" 429 Too Many Requests
INFO:      127.0.0.1:64958 - "POST /predict HTTP/1.1" 429 Too Many Requests
```

Figura 26 – Rate Limiting - 5
Fonte: Elaborado pelo autor (2026)

CORS: Controlando o Acesso de Origens Cruzadas

APIs podem ser consumidas por diversos clientes, como aplicações web em navegadores, que impõem a política de Same-Origin (requisições AJAX/XHR/fetch apenas para o mesmo domínio de origem). Para permitir que origens diferentes acessem a API, é necessário habilitar o CORS (Cross-Origin Resource Sharing).

O CORS é um padrão que permite ao servidor informar ao navegador quais origens externas (domínios) podem acessar seus recursos. Isso é feito através de cabeçalhos HTTP específicos, como Access-Control-Allow-Origin.

Quando habilitar CORS?

- **Necessário:** se sua API for acessada por um frontend (aplicação web) hospedado em um domínio diferente (ex.: API em `api.minhaempresa.com` e frontend em `app.minhaempresa.com`).
- **Opcional/Desnecessário:** se a API for usada apenas em comunicação server-to-server (sem navegador). Desabilitar o CORS nesse cenário aumenta a segurança, prevenindo chamadas de scripts em páginas web.

O FastAPI facilita a configuração do CORS usando um middleware dedicado. Você define explicitamente as origens, métodos HTTP e cabeçalhos permitidos.

Para permitir requisições de um domínio de produção e de um ambiente de desenvolvimento local (localhost), utilizando os métodos GET e POST, o código seria:

```
from fastapi.middleware.cors import CORSMiddleware
from fastapi import FastAPI

app = FastAPI()

# Lista de domínios permitidos
origins_permitidas = [
    "https://app.minhaempresa.com",
    "http://localhost:3000", # Ex: ambiente local de testes do frontend
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins_permitidas,
    allow_credentials=True, # Permite envio de cookies/autenticação
    allow_methods=["GET", "POST"], # Métodos HTTP autorizados
    allow_headers=["*"], # Quais cabeçalhos podem ser enviados (o "*" permite todos)
)
```

Essa configuração fará com que o FastAPI inclua os cabeçalhos CORS necessários nas respostas, autorizando o acesso das origens especificadas.

Boas Práticas Gerais em APIs de ML

Além dos pontos específicos que implementamos, existem várias práticas gerais de segurança e qualidade que devem ser adotadas ao construir APIs (seja para ML como para outros fins).

- **Logs e Monitoramento:** registre as ações da API para detectar erros e ataques. Grave detalhes da requisição (caminho, método, status) sem dados sigilosos (senhas, tokens). Use logs estruturados (JSON) para facilitar a análise. Monitore falhas frequentes.
- **Comunicação Segura (HTTPS):** sempre use HTTPS em produção. Ele criptografa o tráfego para proteger dados sensíveis. O certificado é fácil de obter. Use HTTP só para testes internos.
- **Gerenciamento de Segredos:** nunca deixe senhas ou chaves de acesso no código. Use variáveis de ambiente, arquivos de configuração protegidos (.env não versionado) ou serviços de gerenciamento de segredos. Não exponha informações sensíveis em URLs ou mensagens de erro.
- **Atualização de Dependências:** mantenha suas bibliotecas e frameworks sempre atualizados para corrigir falhas de segurança. Use ferramentas de varredura. A segurança exige atualizações rápidas, mesmo em sistemas funcionando.
- **Princípio do Menor Privilégio:** dê a cada parte do sistema apenas as permissões necessárias. Ex: usuário de banco de dados só com acesso de leitura, se for o bastante. Não rode a aplicação como root desnecessariamente.
- **Testes de Segurança e Hardening:** inclua testes de segurança (com ferramentas como OWASP Zap) no desenvolvimento. Verifique a proteção de endpoints de administração. Em produção, desabilite recursos desnecessários (como documentação automática ou listagem de diretórios) e reforce cabeçalhos HTTP de resposta.

Considerações Essenciais para APIs de Inferência de ML em Produção

Levar uma API de inferência de Machine Learning (ML) à produção exige uma visão que vai além do código, focando na robustez e resiliência da infraestrutura. Os pilares de um serviço de sucesso são: desempenho e resiliência da infraestrutura; segurança e operação contínua.

CONCEITO	DESCRIÇÃO	IMPLEMENTAÇÃO PRÁTICA
Escalabilidade	Capacidade de a API suportar o aumento da demanda (throughput).	Horizontal: Multiplicação de instâncias do serviço com balanceamento de carga. Vertical: Uso de hardware mais potente (CPU/RAM/GPU).
		Uso de containerização (Docker) e orquestradores (Kubernetes, AWS Elastic Beanstalk) com autoscaling. O código deve ser stateless para permitir escalabilidade horizontal sem inconsistências.
Redundância	Eliminação de Pontos Únicos de Falha (SPOF) para garantir a continuidade do serviço em caso de falha.	Mínimo de duas instâncias da API rodando. Redundância em componentes auxiliares (e.g., múltiplos nodes de DB, múltiplas Zonas de Disponibilidade em nuvem).
Alta Disponibilidade (HA)	O resultado da aplicação de redundância e tolerância a falhas, garantindo que o serviço esteja acessível praticamente o tempo todo.	Uso de health-checks e monitoramento para remoção automática de instâncias problemáticas. Implementação de estratégias de deploy (blue-green, canário) para atualizações sem downtime. Planejamento de capacidade para suportar falhas parciais.

Tabela 2 – Conceito, descrição e implementação
Fonte: Elaborado pelo autor (2026)

Em produção, a operação exige atenção a outros fatores cruciais:

- **Observabilidade:** implementação de logs centralizados, métricas de performance e traços para diagnóstico e auditoria.
- **Segurança em Profundidade:** múltiplas camadas de proteção: autenticação na API, Web Application Firewall (WAF) e monitoramento ativo.
- **Controle de Custos:** uso de rate limiting para prevenir abusos e escalabilidade elástica para evitar gastos com capacidade ociosa.
- **Conformidade:** seguir as recomendações de segurança da comunidade, como o OWASP API Security Top 10 (ex.: mitigação de autenticação quebrada, vazamento de dados).

Checklist de Segurança e Boas Práticas para APIs de ML

Antes do go-live da API, verifique se os seguintes requisitos essenciais foram atendidos:

ÁREA	ITEM DE VERIFICAÇÃO
Autenticação e Autorização	Autenticação: Uso de credenciais/tokens (e.g., JWT) para acesso a endpoints sensíveis.
	Autorização: Implementação de controle de acesso para garantir que usuários/chaves só acessem dados/previsões pertinentes.

Validação de Dados	Validação de Entradas: Todos os parâmetros (query, path, body) são validados (tipo, formato, limites). Rejeição de dados inesperados com códigos 400/422.
Comunicação e Exposição	CORS: Configuração correta dos cabeçalhos CORS, permitindo apenas origens confiáveis (ou desabilitado se não for web).
	Transporte Seguro: Disponibilidade exclusiva via HTTPS com certificado válido, incluindo redirecionamento HTTP->HTTPS (e HSTS, se aplicável).
Proteção contra Abuso	Rate Limiting: Implementação de proteção contra uso excessivo (via middleware - e.g., SlowAPI - ou API Gateway), retornando 429 em caso de limite excedido.
Manuseio de Informação	Segredos Protegidos: Chaves, senhas e tokens carregados de forma segura (variáveis de ambiente, serviços de segredo), nunca hardcoded ou em configurações públicas.
	Erros Tratáveis: Respostas de erro que evitam a exposição de detalhes internos (stack traces, versões). Mensagens genéricas para atacantes, mas úteis via logs.
Operação e Manutenção	Logging e Monitoramento: Registro de eventos importantes (acessos, erros, validações falhas) para auditoria. Logs não devem expor informações sensíveis. Alertas de uptime e erros ativos.

	Atualizações e Patches: Dependências mantidas atualizadas e processo estabelecido para a aplicação rápida de patches de segurança.
Robustez em Produção	Infraestrutura Resiliente: Uso de instâncias redundantes, balanceamento de carga e planos de recuperação de desastre. Testes de estresse realizados para estimar a capacidade.
	Revisão Final: Condução de uma revisão de segurança (e.g., testes de penetração leves ou uso de scanners como OWASP Zap) antes da implantação.

Tabela 3 – Requisitos essenciais
Fonte: Elaborado pelo autor (2026)

Cumprir este checklist assegura que a API de inferência estará pronta para operar com segurança, estabilidade e confiabilidade no ambiente de produção. A segurança é um ciclo contínuo de aprendizado e melhoria.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você fortaleceu a API construída nas etapas anteriores adicionando as camadas essenciais de segurança e robustez que qualquer serviço de Machine Learning precisa para operar em produção. Primeiro, exploramos como proteger endpoints sensíveis com API Keys e tokens JWT, entendendo as diferenças entre autenticação e autorização, além dos riscos de deixar uma API de predição exposta sem controle de acesso.

Na sequência, você implementou validação de dados com Pydantic, discutiu ataques comuns por entradas malformadas e configurou mecanismos de proteção operacional, como rate limiting para conter abusos.

Por fim, ampliamos a visão para o ambiente real de produção, revisando boas práticas como uso de HTTPS, gestão de segredos, logs estruturados, monitoramento e princípios de infraestrutura resiliente.

O objetivo foi mostrar que expor um modelo não é apenas “colocar o endpoint no ar”, mas construir um serviço confiável, seguro e preparado para escalar.

REFERÊNCIAS

GÉRÓN, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. Beijing: O'Reilly Media, 2023.

GIFT, N.; DEZA, A. **Practical MLOps**: Operationalizing Machine Learning Models. Sebastopol: O'Reilly Media, 2021.

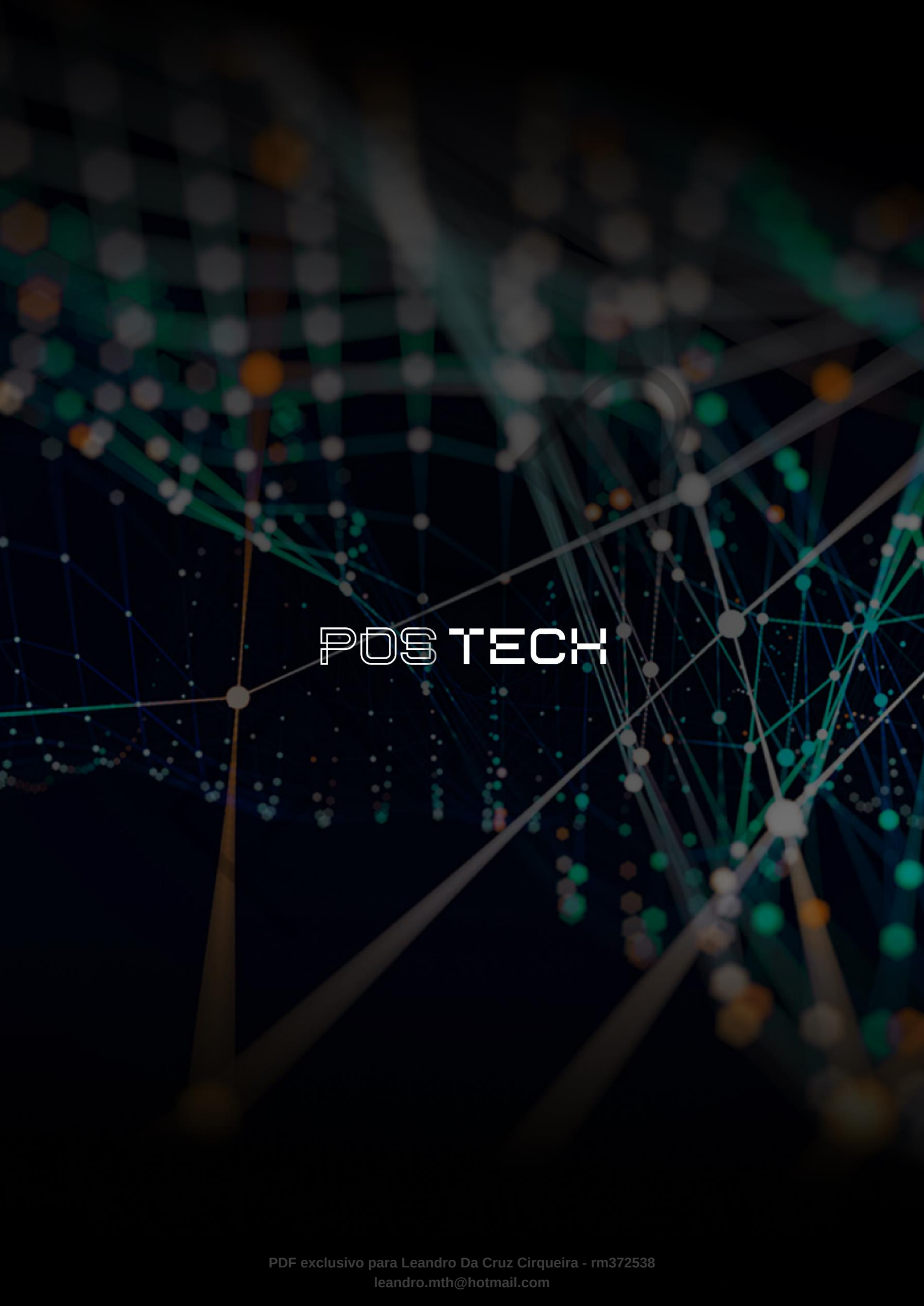
LUBANOVIC, B. **FastAPI**: Modern Python Web Development. Sebastopol: O'Reilly Media, 2023.

NEWMAN, S. **Building Microservices**: Designing Fine-Grained Systems. Sebastopol: O'Reilly Media, 2021.

PALAVRAS-CHAVE

JWT. Token. API. WEB HTTP. REST. API RESTful. GraphQL. gRPC. JSON. MLOps.

EMEND



POSTECH